We expose the service on webpack-dev-server's default port of 3035 (line 20).

We want the webpack-dev-server to pick up and recompile our changes automatically as we develop locally without having to restart. That's why on line 22, like in our web service, we mount our local files into the container.

The webpack-dev-server command expects to be run with the same configuration as our Rails app. Luckily, having extracted these into files, we can simply reuse the same env_files (lines 23–25).

However, to ensure that webpacker-dev-server responds to requests from any IP address, we set WEBPACKER_DEV_SERVER_HOST to 0.0.0.0 (line 27), much like we did with the Rails server.

Having configured our webpack_dev_server service, we also need to set a Rails environment variable for our web service, so it knows where to find the webpack-dev-server (line 14).

Now we need to start our web service to use our new image and pick up these config changes:

```
$ docker-compose up -d web
```

and then launch our new service:

```
$ docker-compose up -d webpack_dev_server
```

A Hello World React App

Our purpose here is to ensure we can configure our Rails app, using Docker, to allow us to develop modern JavaScript apps using technologies like React. To that end, we just need to show that a simple React app is compiled and loads correctly with our setup.

When we install Webpacker, it adds a sample "Hello World" React app in app/javascript/packs/hello_react.jsx that renders a saying "Hello React!":

```
import React from 'react'
import ReactDOM from 'react-dom'
import PropTypes from 'prop-types'

const Hello = props => (
  <div>Hello {props.name}!</div>
)

Hello.defaultProps = {
  name: 'World'
}
```